

ASR Based Transcription and Transliteration System

Task 2 Project Report · Whisper ASR + Indic Transliteration + Gradio Interface

Gourilakshmi S

Overview

This project implements an end to end deployable system for Tamil speech transcription and Indic script transliteration. Audio input is transcribed using OpenAI Whisper with forced Tamil language decoding, converted between Indic scripts via indic transliteration, and served through a Gradio web interface. The system is containerised with Docker and backed by a queue based audio buffer for chunked processing.

ASR Model	Transliteration	Interface	Deployment
openai/whisper small	indic transliteration v2.3.x	Gradio v6.x port 7860	Docker + docker compose

Audio Input	Sample Clips	Supported Scripts	Buffer Strategy
.mp3 / .wav / mic	5 Tamil gTTS clips	9 (Tamil, Malayalam, Devanagari...)	queue.Queue 30s chunks

Models Used

Component	Model / Library	Tamil Support	Role
ASR (primary)	openai/whisper small	Native multilingual	Tamil transcription
ASR (alt)	openai/whisper medium	Native multilingual	Higher accuracy
Transliteration	indic transliteration v2.3.82	All Indic scripts	Cross script conversion
Interface	Gradio v6.14.0	N/A	Web UI on port 7860

Part A ASR Transcription Pipeline

Whisper is loaded via HuggingFace Transformers and evaluated on 5 Tamil gTTS audio clips. The critical design decision is the use of `forced_decoder_ids`: the processor generates language token `<|ta|>` and task token `<|transcribe|>` which are injected at the start of every generation sequence. Without this, Whisper auto detects the language and on short Tamil clips it frequently misidentifies the script as Telugu, Sinhala, or Malayalam.

Key Findings

- `forced_decoder_ids` is load bearing infrastructure, analogous to IndicTrans2 IndicProcessor in Task 1. The model and its language tokens are a system; removing them degrades output severely.
- whisper small achieves acceptable transcription on gTTS generated Tamil. whisper medium produces noticeably better output on conversational speech with background noise.
- The pipeline supports both full file transcription (single pass) and chunked transcription (via AudioBufferManager), selectable at runtime via the Gradio interface.

Part B Buffer System and Chunked Processing

The `AudioBufferManager` implements a FIFO queue using Python's `queue.Queue(maxsize=20)`. Audio files are split into 30 second chunks (480,000 samples at 16kHz) matching Whisper's native input window. The final chunk is zero padded to exactly 30 seconds. Silence does not affect transcription because Whisper detects speech boundaries internally.

Processing a 75 second Tamil clip

Processing Mode	Chunks Created	Final Chunk Handling	Output
Direct (no buffer)	1 (full file)	N/A	Single transcript
Chunked buffer	3 chunks (30s + 30s + 15s)	Zero padded to 30s	Concatenated transcript

Key Findings

- Zero padding the final chunk is mandatory. Whisper's processor expects a fixed size input tensor. Passing a short final chunk without padding raises a shape mismatch error.
- Overflow protection: `drop_old_chunk()` uses `put_nowait()` and catches `queue.Full`, dropping the oldest chunk rather than blocking. For real time streaming, stale audio is less valuable than fresh audio.
- Thread safety: a `threading.Lock()` protects the `clear()` operation. The queue itself is thread safe by design.

Part C Transliteration Engine and Cross Script Conversion

No model inference. The transliteration engine operates purely on the text output from Part A. Nine target scripts are supported. The central engineering challenge was cross Indic conversion. Direct Tamil to Malayalam transliteration produces garbled output because the indic transliteration library cannot map directly between two Brahmic script families without phonemic disambiguation.

ITRANS Bridge Pattern

All cross Indic conversions are routed through ITRANS (a lossless romanized intermediate) as a two step bridge: Tamil to ITRANS to Target script. ITRANS encodes all phonemic information in a script neutral form, enabling correct round trip conversion between any two Indic scripts.

Transliteration of வணக்கம் (vanakkam, hello)

Target Script	Output	Bridge Used	Correct
latin (ITRANS)	vaNakkam	Direct	Yes
malayalam	വണക്കം	ITRANS bridge	Yes (after fix)
devanagari	वणक्कम्	ITRANS bridge	Yes
telugu	వణక్కమ్	ITRANS bridge	Yes

Key Findings

- The ITRANS bridge is the transliteration equivalent of IndicTrans2 IndicProcessor. A non negotiable preprocessing step that the library cannot skip without producing incorrect output.
- Tamil combining marks (virama U+0BCD, vowel signs) are preserved correctly through the ITRANS intermediate. The lossless encoding prevents the Unicode fragmentation seen in naive direct conversion.

- latin and itrans map to the same sanscript constant (ITRANS). Both produce romanized output, making them interchangeable for Latin script display.

Conclusion

Across all three parts, a single architectural insight holds: **intermediate representations are load bearing**. Just as IndicTrans2 requires IndicProcessor and Whisper requires forced_decoder_ids, the transliteration engine requires the ITRANS bridge for cross Indic output. In each case, the model and its pre and post processing pipeline are a single system. Removing any one component degrades the entire output. The buffer system's zero padding requirement follows the same logic: Whisper's tensor shapes are fixed contracts that the surrounding code must respect. The system demonstrates that Tamil NLP engineering at every layer, ASR, tokenization, transliteration, rewards those who understand the script's phonemic and Unicode structure.